

Practical Task - 5

2520080005

TASK - 1:

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Creating server.c

```
LST@LAPTOP-EK5DCJVE:~# mkdir practical_tasks
LST@LAPTOP-EK5DCJVE:~# cd practical_tasks
LST@LAPTOP-EK5DCJVE:~/practical_tasks# nano server.c
LST@LAPTOP-EK5DCJVE:~/practical_tasks# |
```

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#define SERVER_FIFO "/tmp/server_fifo"
typedef struct {
```

```

    pid_t pid;

    char message[200];
} Request;

void process_client(Request req)
{
    char client_fifo[100];
    char response[300];
    sprintf(client_fifo, "/tmp/client_%d_fifo", req.pid);
    printf("Processing Client %d\n", req.pid);

    printf("Message: %s\n", req.message);
    sprintf(response,
        "Server received your message: %s",
        req.message);
    int fd = open(client_fifo, O_WRONLY);
    if (fd == -1) {
        perror("open client FIFO");
        return;
    }
    write(fd, response, strlen(response) + 1);
    close(fd);
}

int main()
{
    Request req;
    unlink(SERVER_FIFO);
    if (mkfifo(SERVER_FIFO, 0666) == -1) {
        perror("mkfifo");
        return 1;
    }
    printf("=====\n");
    printf("    SERVER STARTED\n");

```

```

printf("=====\n");

printf("Server FIFO: %s\n", SERVER_FIFO);

printf("Waiting for clients...\n\n");

int fd = open(SERVER_FIFO, O_RDONLY);

if (fd == -1) {

    perror("open server FIFO");

    unlink(SERVER_FIFO);

    return 1;

}

while (1)

{

    ssize_t n = read(fd, Creq, sizeof(req));

    if (n == sizeof(req))

    {

        pid_t pid = fork();

        if (pid == 0)

        {

            process_client(req);

            exit(0);

        }

    }

}

close(fd);

unlink(SERVER_FIFO);

return 0;

```

compilation:

```
root@LAPTOP-4H877K4U:~# mkdir ~/practical_task
root@LAPTOP-4H877K4U:~# cd ~/practical_task
root@LAPTOP-4H877K4U:~/practical_task# nano server.c
root@LAPTOP-4H877K4U:~/practical_task# nano server.c.
root@LAPTOP-4H877K4U:~/practical_task# nano server.c
root@LAPTOP-4H877K4U:~/practical_task# gcc server.c -o server
root@LAPTOP-4H877K4U:~/practical_task# ls
server  server.c  server.c.
root@LAPTOP-4H877K4U:~/practical_task# |
```

CLIENT

Create client.c

```
LST@LAPTOP-EK5DCJVE:~# cd practical_tasks
LST@LAPTOP-EK5DCJVE:~/practical_tasks# nano client.c
LST@LAPTOP-EK5DCJVE:~/practical_tasks# |
```

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
```

```
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#define SERVER_FIFO "/tmp/server_fifo"
typedef struct {
    pid_t pid;
    char message[200];
} Request;

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        printf("Usage: %s \"message\\n\\n\", argv[0]);
        return 1;
    }
    pid_t pid = getpid();
    char client_fifo[100];
    sprintf(client_fifo, "/tmp/client_%d_fifo", pid);
    if (mkfifo(client_fifo, 0666) == -1)
    {
        perror("mkfifo");
        return 1;
    }
    Request req;
    req.pid = pid;
    strcpy(req.message, argv[1]);
    int server_fd = open(SERVER_FIFO, O_WRONLY);
    if (server_fd == -1)
    {
```

```

    perror("Server is not running");
    unlink(client_fifo);
    return 1;
}
write(server_fd, Creq, sizeof(req));
close(server_fd);
int client_fd = open(client_fifo, O_RDONLY);
if (client_fd == -1)
{
    perror("open client FIFO");
    unlink(client_fifo);
    return 1;
}
char response[300];
read(client_fd, response, sizeof(response));
printf("\n=====\\n");
printf("    SERVER RESPONSE\\n");
printf("=====\\n");
printf("%s\\n\\n", response);
close(client_fd);
unlink(client_fifo);
return 0;
}

```

Compilation

```
root@LAPTOP-4H877K4U: ~/ | X + v
root@LAPTOP-4H877K4U:~# cd ~/practical_task
root@LAPTOP-4H877K4U:~/practical_task# nano client.c
root@LAPTOP-4H877K4U:~/practical_task# gcc client.c -o client
root@LAPTOP-4H877K4U:~/practical_task# ls
client client.c server server.c server.c.
root@LAPTOP-4H877K4U:~/practical_task# |
```

Run Client

```
Ubuntu x + v - □ X
LST@LAPTOP-EK5DCJVE:~# cd practical_tasks
LST@LAPTOP-EK5DCJVE:~/practical_tasks# ./server
=====
SERVER STARTED
=====
Server FIFO: /tmp/server_fifo
Waiting for clients...

Processing Client 1376
Message: Hello from Client 1
Processing Client 1388
Message: Hello from Client 2
|

Ubuntu x + v - □ X
LST@LAPTOP-EK5DCJVE:~# cd practical_tasks
LST@LAPTOP-EK5DCJVE:~/practical_tasks# nano client.c
LST@LAPTOP-EK5DCJVE:~/practical_tasks# gcc client.c -o client
LST@LAPTOP-EK5DCJVE:~/practical_tasks# ./client
Usage: ./client "message"
LST@LAPTOP-EK5DCJVE:~/practical_tasks# ./client "Hello from Client 1"
=====
SERVER RESPONSE
=====
Server received your message: Hello from Client 1

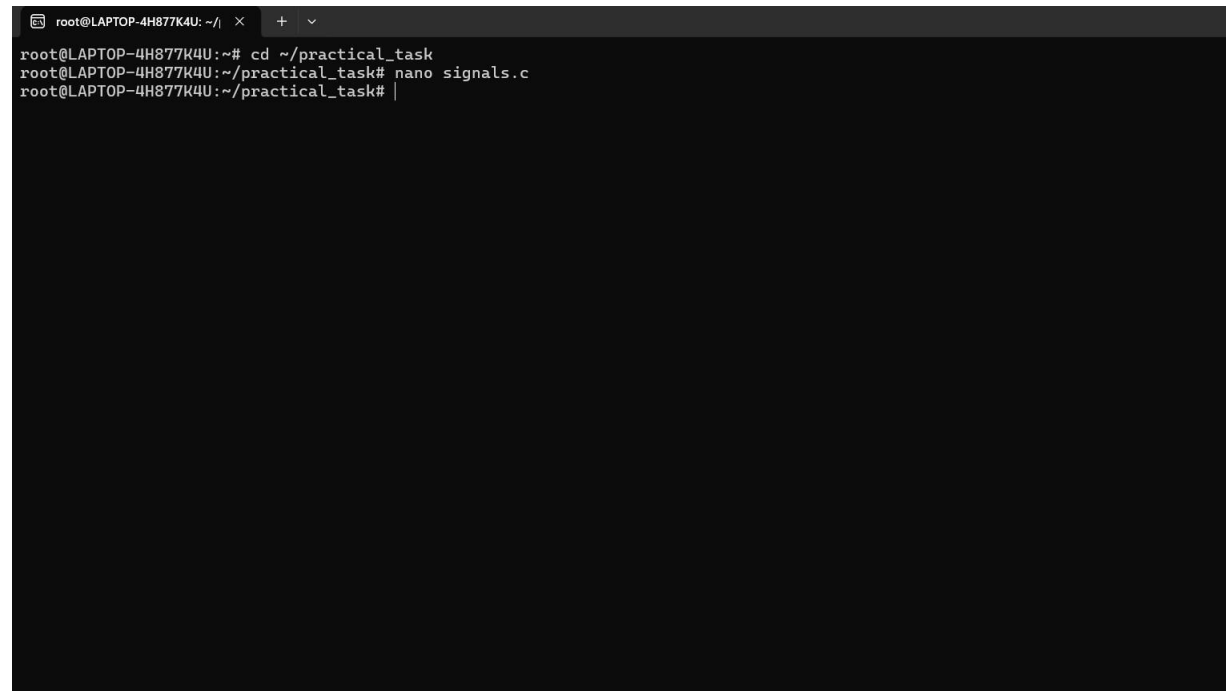
LST@LAPTOP-EK5DCJVE:~/practical_tasks# ./client "Hello from Client 2"
=====
SERVER RESPONSE
=====
Server received your message: Hello from Client 2

LST@LAPTOP-EK5DCJVE:~/practical_tasks# |
```

TASK 2 — POSIX SIGNAL HANDLING

2. Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

Create signals.c

A terminal window with a dark background and light text. The window title is 'root@LAPTOP-4H877K4U: ~/ | x + v'. The terminal shows the following commands and output:

```
root@LAPTOP-4H877K4U:~# cd ~/practical_task
root@LAPTOP-4H877K4U:~/practical_task# nano signals.c
root@LAPTOP-4H877K4U:~/practical_task# |
```

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void signal_handler(int sig)
{
    if (sig == SIGINT)
    {
        printf("\nSIGINT received - Interrupt signal\n");
    }
}
```

```

else if (sig == SIGTERM)
{
    printf("\nSIGTERM received - Termination signal\n");
}
else if (sig == SIGUSR1)
{
    printf("\nSIGUSR1 received - User defined signal\n");
}

fflush(stdout);
}

int main()
{
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("=====\n");
    printf("  POSIX SIGNAL HANDLING\n");
    printf("=====\n");
    printf("Process ID (PID): %d\n", getpid());
    printf("Waiting for signals...\n");
    while (1)
    {
        pause();
    }
    return 0;
}

```

Compilation:

```
root@LAPTOP-4H877K4U: ~/ | x + v
root@LAPTOP-4H877K4U:~# cd ~/practical_task
root@LAPTOP-4H877K4U:~/practical_task# nano signals.c
root@LAPTOP-4H877K4U:~/practical_task# gcc signals.c -o signals
root@LAPTOP-4H877K4U:~/practical_task# ls
client client.c server server.c server.c. signals signals.c
root@LAPTOP-4H877K4U:~/practical_task# ./signals
=====
POSIX SIGNAL HANDLING
=====
Process ID (PID): 3583
Waiting for signals...
|
```

```
root@LAPTOP-4H877K4U: ~/ | x + v - □ x
root@LAPTOP-4H877K4U:~# cd ~/practical_task
root@LAPTOP-4H877K4U:~/practical_task# nano signals.c
root@LAPTOP-4H877K4U:~/practical_task# gcc signals.c -o signals
root@LAPTOP-4H877K4U:~/practical_task# ls
client client.c server server.c server.c. signals signals.c
root@LAPTOP-4H877K4U:~/practical_task# ./signals
=====
POSIX SIGNAL HANDLING
=====
Process ID (PID): 3583
Waiting for signals...

root@LAPTOP-4H877K4U: ~/ | x + v - □ x
root@LAPTOP-4H877K4U:~# cd ~/practical_task
root@LAPTOP-4H877K4U:~/practical_task# kill -SIGUSR1 2500
-bash: kill: (2500) - No such process
root@LAPTOP-4H877K4U:~/practical_task# |
```